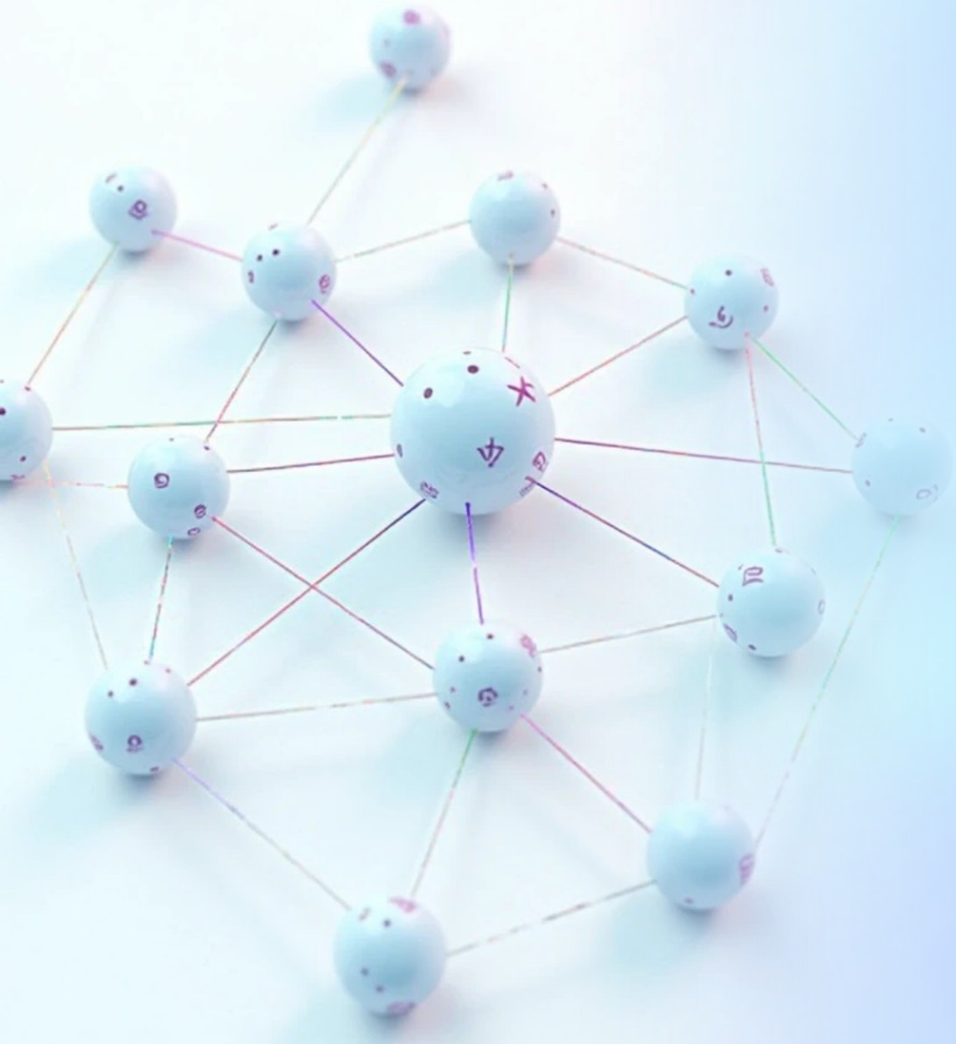
Introduction to Graph Data Structure

By Team 1



1. What is Graph Data Structure?

Graph is a non-linear data structure consisting of vertices and edges. The vertices are sometimes also referred to as nodes and the edges are lines or arcs that connect any two nodes in the graph. More formally a Graph is composed of a set of vertices (V) and a set of edges (E). The graph is denoted by G(V, E).

2. Components of Graph in Data Structures

A graph is made up of several key components that define its structure and relationships.

The main components are:



3. Vertices (Nodes)

1

Definition

Vertices are the fundamental units of a graph, representing entities or data points.

2

Example

In a social network graph, each person is a vertex.

3

Notation

Usually denoted by V(G) or simply V.



4. Edges (Links)

Definition

Edges are the connections or links between vertices that show relationships or associations.

Example

In a road map, an edge represents a road connecting two cities.

Notation

Represented by E(G) or simply E, where each edge connects two vertices (u, v).



5. Weight (or Cost)

1

Definition

A weight or cost is a numerical value assigned to an edge. It usually represents distance, time, or cost associated with moving between two vertices.

2

Example

In a flight network, the weight may represent the flight distance or fare.

3

Used in

Weighted graphs (e.g., Dijkstra’s Algorithm, Minimum Spanning Tree).



6. Direction

Definition

Indicates whether edges have a specific direction (from one vertex to another).

- Types:
  - Directed Graph (Digraph): Edges have direction (A → B).
  - Undirected Graph: Edges have no direction (A — B).



7. Degree of a Vertex

1

Definition

The degree of a vertex is the number of edges incident to it.

- Types:
  - In-degree: Number of incoming edges (in directed graphs).
  - Out-degree: Number of outgoing edges (in directed graphs).

2

Example

If vertex A has 2 incoming and 3 outgoing edges → In-degree = 2, Out-degree = 3.

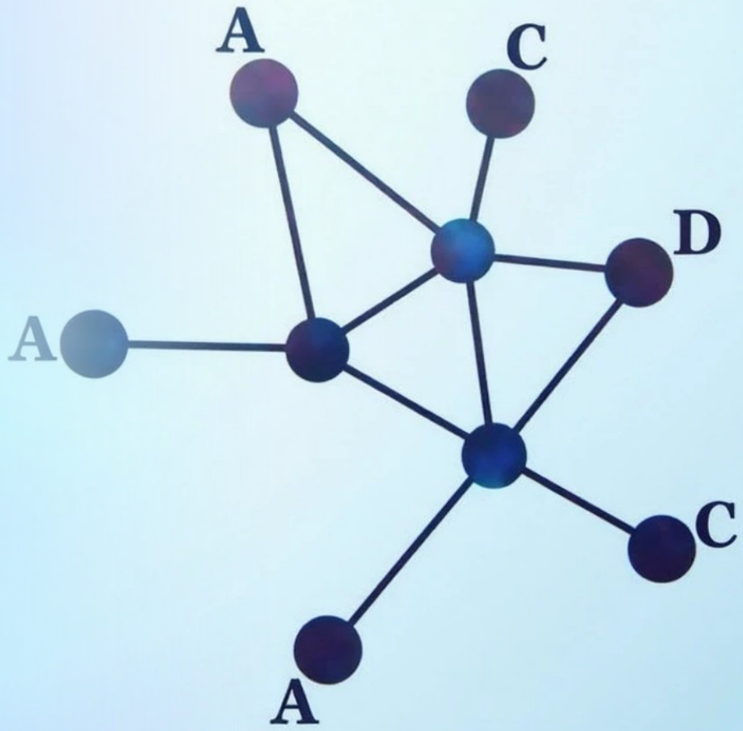
8. Path

Definition

A path is a sequence of vertices connected by edges.

Example

A → B → C → D is a path from vertex A to D.





9. Cycle

1 Definition

A cycle occurs when a path starts and ends at the same vertex.

2 Example

A → B → C → A forms a cycle.

10. Connected Components

Definition

A connected component is a subgraph in which every pair of vertices is connected by a path, and no vertex is connected to any vertex outside this subgraph.

Example

If a graph has two disconnected parts, each part is a separate connected component.



11. Subgraph

1

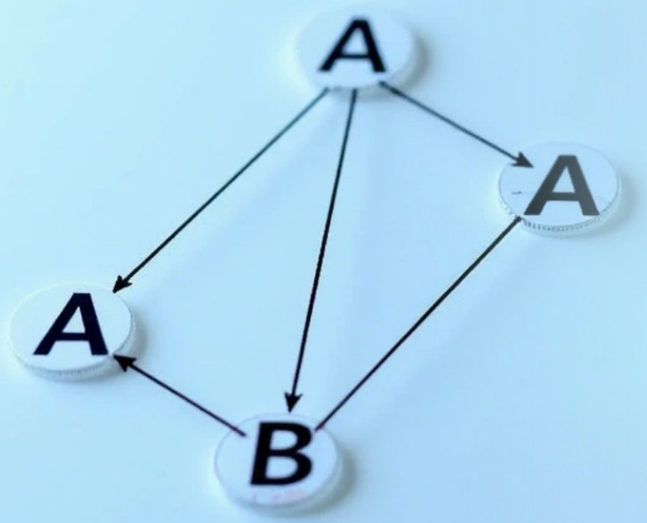
Definition

A subgraph is a smaller portion of a graph formed using a subset of vertices and edges from the main graph.

2

Example

A city network where only a few cities and roads are selected forms a subgraph.



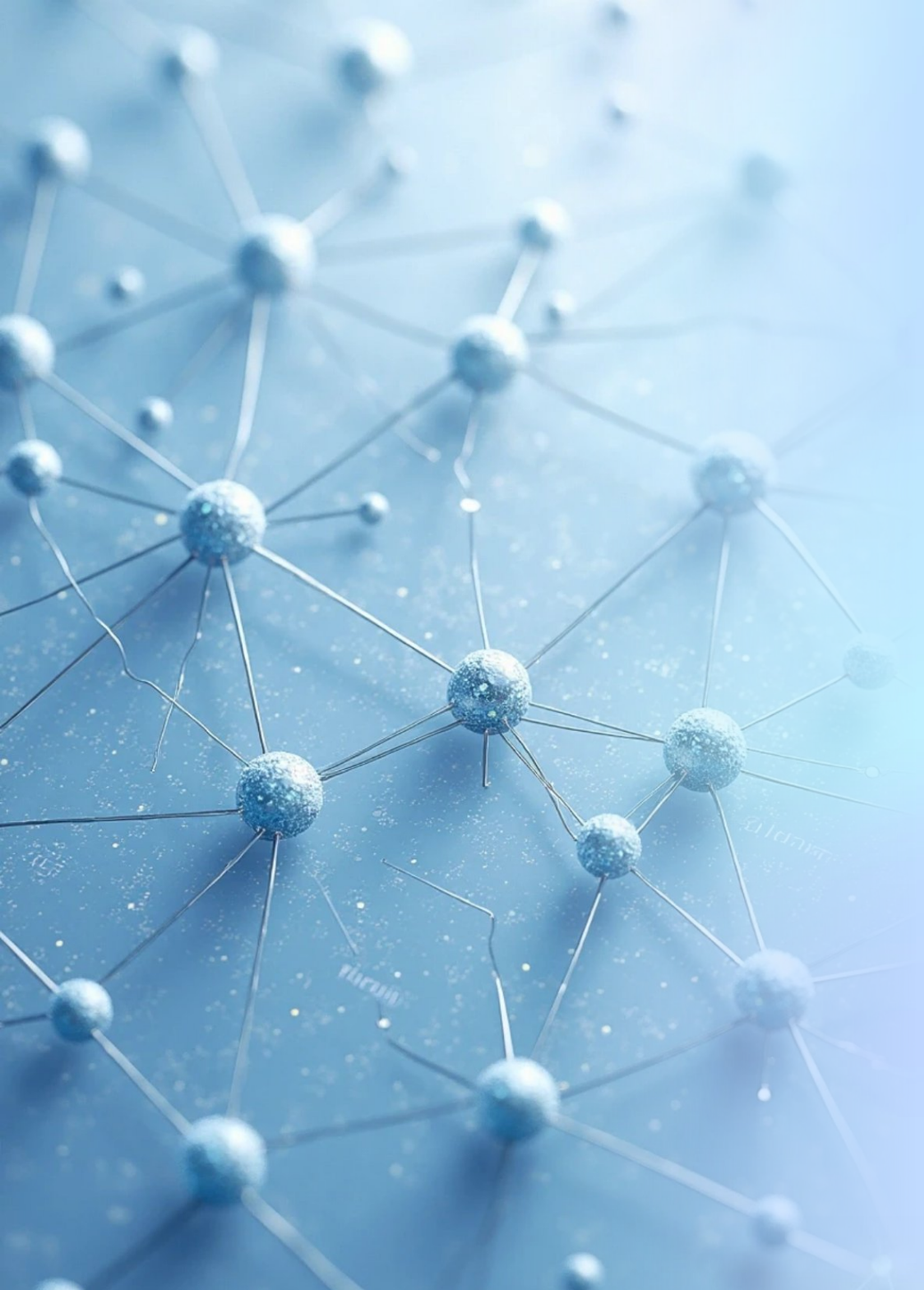
12. Adjacency

Definition

Two vertices are said to be adjacent if they are directly connected by an edge.

Example

In graph G, if there is an edge between A and B, then A and B are adjacent vertices.



13. Advantages of Graph Data Structure (1)

1

Represents Complex Relationships

- Graphs can show how different items are connected to each other.
- Unlike trees or arrays, graphs can have loops and multiple connections.

2

Real-World Applications

- Graphs are used in real-life problems like maps (shortest path), social networks (friend connections), and the internet (web links).

14. Advantages of Graph Data Structure (2)

Powerful Algorithms Available

- Many ready-to-use algorithms help solve graph problems efficiently:
  - BFS/DFS – for searching and traversing
  - Dijkstra’s Algorithm – for finding shortest paths
  - Minimum Spanning Tree – for connecting all points with minimum cost
  - Topological Sorting – for ordering tasks or dependencies

Easy to Model Complex Data

- Graphs make it easier to understand and visualize complicated systems like traffic networks or recommendation systems.

15. Disadvantages of Graph Data Structure (1)

1

Hard to Understand for Beginners

• Graph concepts and algorithms can be confusing for people new to the topic.

2

High Memory Usage

• Storing large graphs can take a lot of memory, especially if there are many connections.





16. Disadvantages of Graph Data Structure (2)

Complex Implementation

- Writing and debugging graph algorithms can be tricky and time-consuming.

Difficult Visualization

- When a graph becomes large, it’s hard to draw or analyze it clearly.



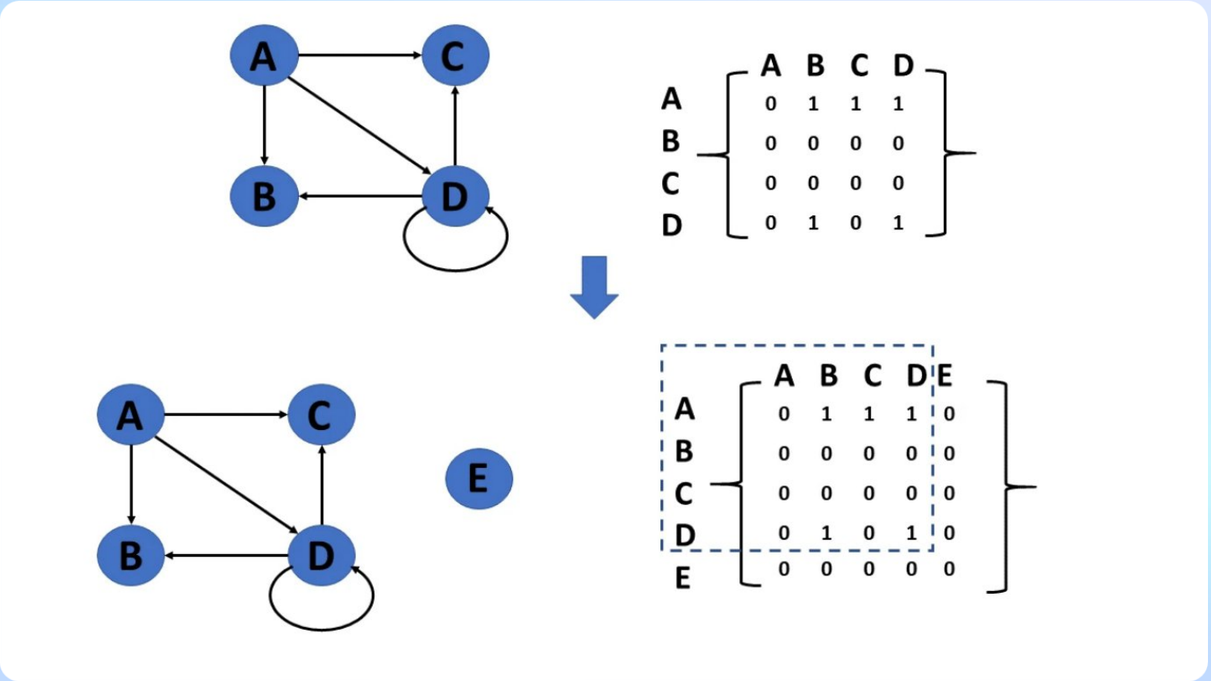
17. Operations on Graphs in Data Structures

The operations you perform on the graphs in data structures are listed below:

1. Creating graphs
2. Insert vertex
3. Delete vertex
4. Insert edge
5. Delete edge

18. Insert Vertex

When you add a vertex that after introducing one or more vertices or nodes, the graph’s size grows by one, increasing the matrix’s size by one at the row and column levels.



19. Delete Vertex

1

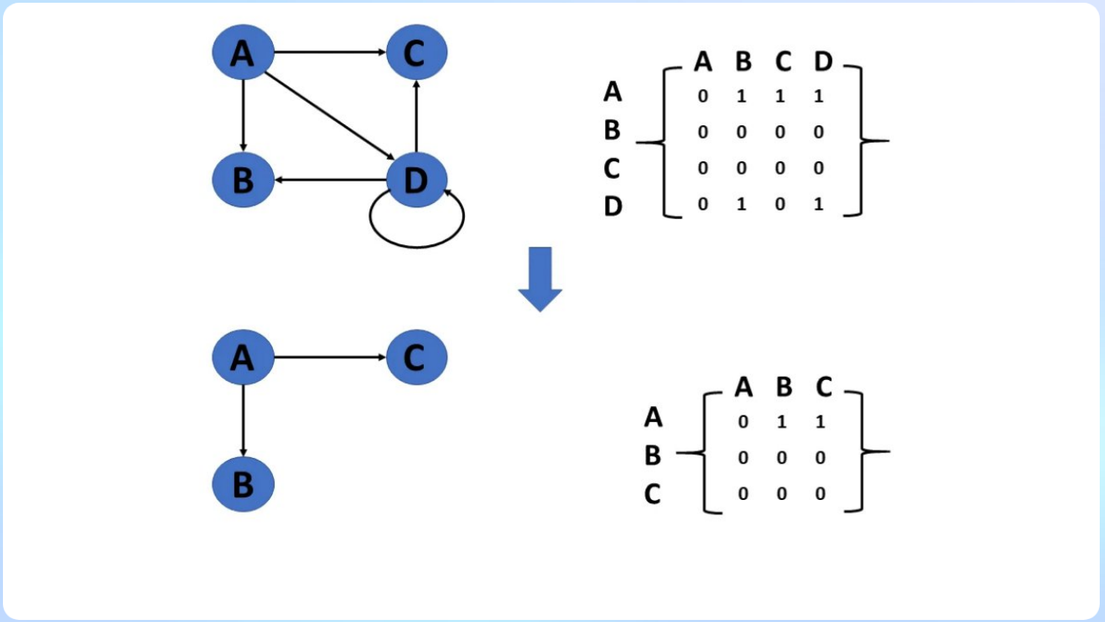
Deleting a vertex refers to removing a specific node or vertex from a graph that has been saved.

2

If a removed node appears in the graph, the matrix returns that node.

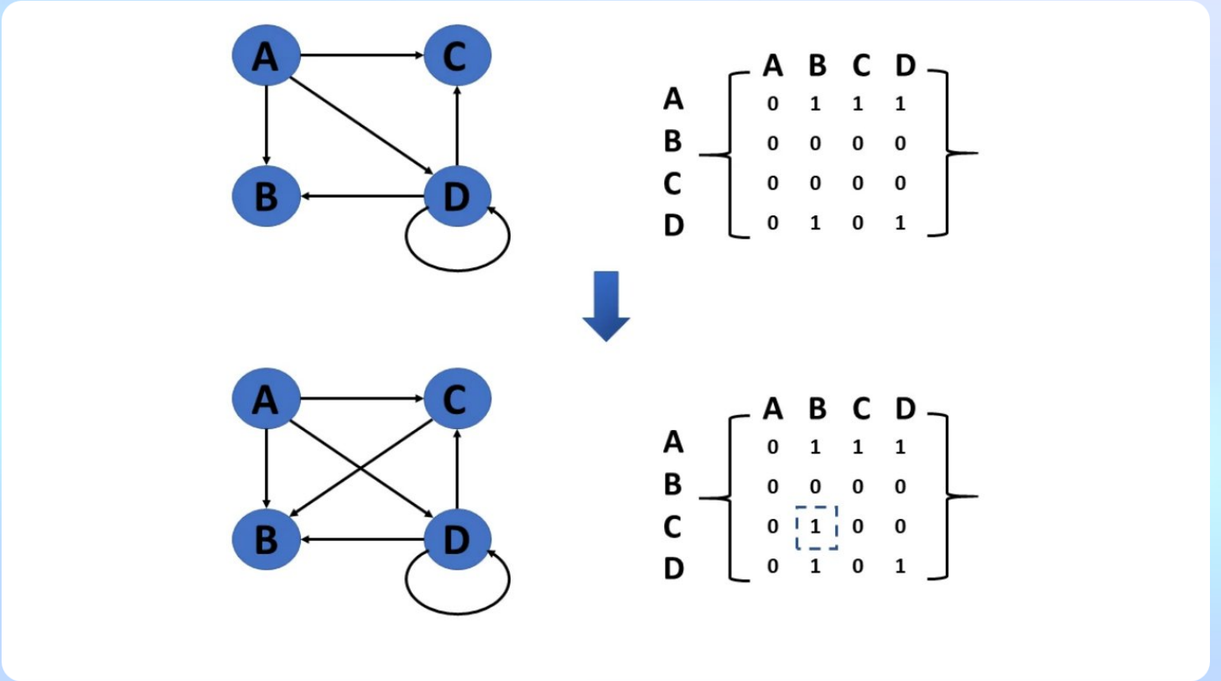
3

If a deleted node does not appear in the graph, the matrix returns the node not available.



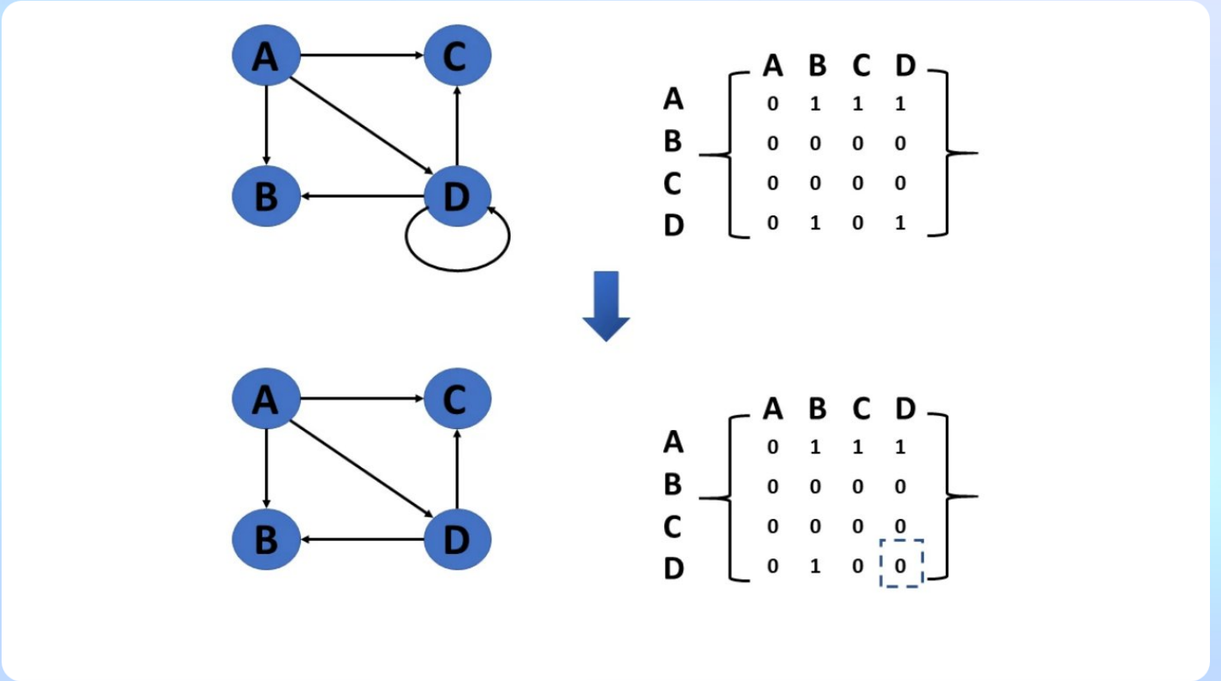
20. Insert Edge

Connecting two provided vertices can be used to add an edge to a graph.



21. Delete Edge

The connection between the vertices or nodes can be removed to delete an edge.



Thank You